

Embedded Systems A5: Wireless Motor PWM
Final Project Report

Course: EEL 4740/6741 - Embedded Systems - Spring 2026

Instructor: Dr. Alexandro Castellanos, Ph.D

Group Members:

Ryan Saji - U11496338

Mayaz Ahmed Bin Rouf - U35999688

Ayush Parida- U04805358

Date: 05/01/2026

Objective

The objective of this project is to design and implement a wireless motor speed control system using a smartphone as the user interface. The user inputs a speed value between 0% and 100% through a Bluetooth application. This command is transmitted wirelessly to an HM-11 Bluetooth Low Energy (BLE) module, which forwards the data to the MSP430FR2355 microcontroller via UART communication.

The MSP430 processes the received input and generates a corresponding Pulse Width Modulation (PWM) signal using its Timer_B peripheral. This PWM signal is then used to control the speed of a 12V DC motor through an L298N motor driver. At the same time, a 16×2 I2C LCD is used to display the current speed percentage, providing real-time feedback to the user.

This project demonstrates the integration of wireless communication, serial communication, I2C interfacing, PWM generation, and motor control within an embedded system.

Key Requirements

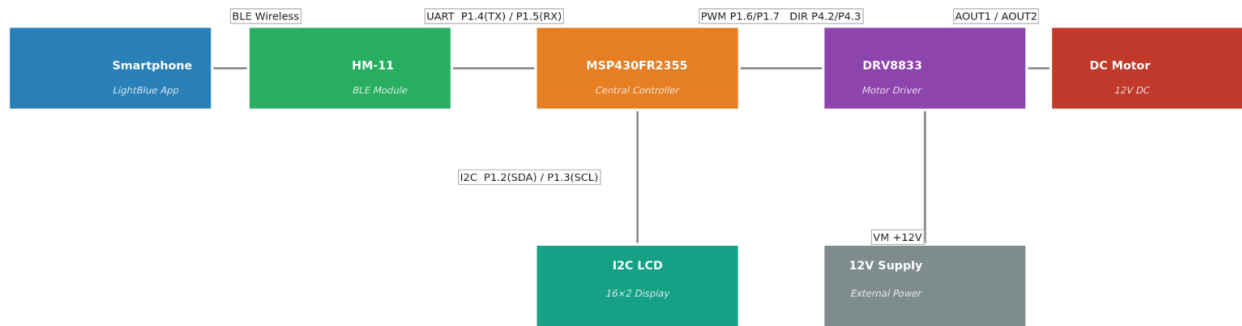
- Wireless communication between smartphone and MSP430FR2355 using HM-11 BLE module
- UART communication between HM-11 and MSP430FR2355
- PWM generation using Timer_B3 on MSP430FR2355
- Motor control using L298N motor driver
- Speed control range from 0% to 100%
- Real-time speed display using I2C LCD
- Stable system operation with common ground between all components
- External 12V supply for motor operation

System Overview Block Diagram

The system consists of four main components:

1. Smartphone (LightBlue App) - sends speed commands via Bluetooth
2. HM-11 BLE Module - receives wireless data and forwards it via UART
3. MSP430FR2355 Microcontroller - processes data and generates PWM
4. L298N Motor Driver + DC Motor - controls motor speed based on PWM

Additionally, an I2C LCD displays the motor speed.



Pseudocode

```
BEGIN
```

```
INITIALIZE SYSTEM:
```

```
    Configure PWM on P6.1 using Timer_B3
    Configure UART on P1.6 (TX) and P1.7 (RX)
    Configure I2C LCD (P1.2 SDA, P1.3 SCL)
    Set motor speed to 0%
```

```
MAIN LOOP:
```

```
    Wait for Bluetooth input
```

```
UART RECEIVE INTERRUPT:
```

```
    Read incoming character
```

```
    IF character is digit:
```

```
        Append to temporary variable
```

```
    IF character is '*':
```

```
        Clear speed
        Set PWM = 0
        Update LCD
```

```
    IF character is '#':
```

```
        Convert input to integer
        Clamp between 0-100
        Update PWM duty cycle
        Update LCD display
```

```
END
```

C Code

```

/
#include <msp430.h> #include "LiquidCrystal_I2C.h"
/**

```

- Bluetooth Motor Control + LCD Display

- HM-11 + MSP430FR2355 + L298N

```
*/
volatile unsigned int temp = 0; volatile unsigned int speed = 0; volatile unsigned char new_speed_flag = 0; volatile unsigned char
clear_flag = 0;
void UART_Init(void); void LCD_ShowSpeed(unsigned int s);
int main(void) { WDTCTL = WDTPW | WDTHOLD;
// PWM output P6.1
P6DIR |= BIT1;
P6SEL0 |= BIT1;

// UART pins P1.6 TX, P1.7 RX
P1SEL0 |= BIT6 | BIT7;
P1SEL1 &= ~(BIT6 | BIT7);

PM5CTL0 &= ~LOCKLPM5;

// LCD
I2C_Init(0x27);
LCD_Setup();
LCD_ClearDisplay();

LCD_SetCursor(0,0);
LCD_Write("BT Motor PWM ");
LCD_SetCursor(0,1);
LCD_Write("Speed: 0% ");

// PWM Timer
TB3CCR0 = 655;
TB3CCTL2 = OUTMOD_7;
TB3CCR2 = 0;
TB3CTL = TBSSEL_1 | MC_1 | TBCLR;

UART_Init();
__enable_interrupt();

while(1)
{
    if(clear_flag)
    {
        clear_flag = 0;
        temp = 0;
        speed = 0;
        TB3CCR2 = 0;
        LCD_ShowSpeed(0);
    }

    if(new_speed_flag)
    {
        new_speed_flag = 0;

        if(speed > 100)
        {
            speed = 100;
        }
    }
}
```

```

    }

    TB3CCR2 = (speed * 655) / 100;
    LCD_ShowSpeed(speed);
}

// Small delay to prevent LCD from being hammered too fast
__delay_cycles(10000);
}

return 0;

}

void UART_Init(void) { UCA0CTLW0 |= UCSWRST; UCA0CTLW0 |= UCSSEL__SMCLK;
// 9600 baud assuming 1 MHz SMCLK
UCA0BRW = 6;
UCA0MCTLW = UCOS16 | UCBRF_8 | 0x2000;

UCA0CTLW0 &= ~UCSWRST;
UCA0IE |= UCRXIE;

}

void LCD_ShowSpeed(unsigned int s) { // Disable UART RX while writing to LCD UCA0IE &= ~UCRXIE;
LCD_SetCursor(0,0);
LCD_Write("Motor Speed  ");

LCD_SetCursor(0,1);
LCD_Write("Speed: ");

if(s == 100)
{
    LCD_Write("100%  ");
}
else if(s >= 10)
{
    LCD_WriteNum(s);
    LCD_Write("%  ");
}
else
{
    LCD_WriteNum(s);
    LCD_Write("%  ");
}

UCA0IE |= UCRXIE;

}

#pragma vector = EUSCI_A0_VECTOR __interrupt void EUSCI_A0_ISR(void) { char c;
if(UCA0IFG & UCRXIFG)
{
    c = UCA0RXBUF;

    if(c >= '0' && c <= '9')

```

```

{
    temp = temp * 10 + (c - '0');

    if(temp > 100)
    {
        temp = 100;
    }
}
else if(c == '*')
{
    clear_flag = 1;
    temp = 0;
}
else if(c == '#')
{
    speed = temp;
    temp = 0;
    new_speed_flag = 1;
}
}

}

// — End of Code —————

```

Wiring Diagram

The MSP430FR2355 acts as the central controller and connects to all components:

- UART communication connects the HM-11 module
- I2C communication connects the LCD
- PWM output connects to the motor driver
- External power drives the motor

All grounds are connected together to ensure proper reference voltage.

Pin Connection Summary

MSP430FR2355 → HM-11 BLE Module

MSP430	HM-11
P1.7 (RX)	TXD
P1.6 (TX)	RXD
5V	VCC
GND	GND

MSP430FR2355 → I2C LCD

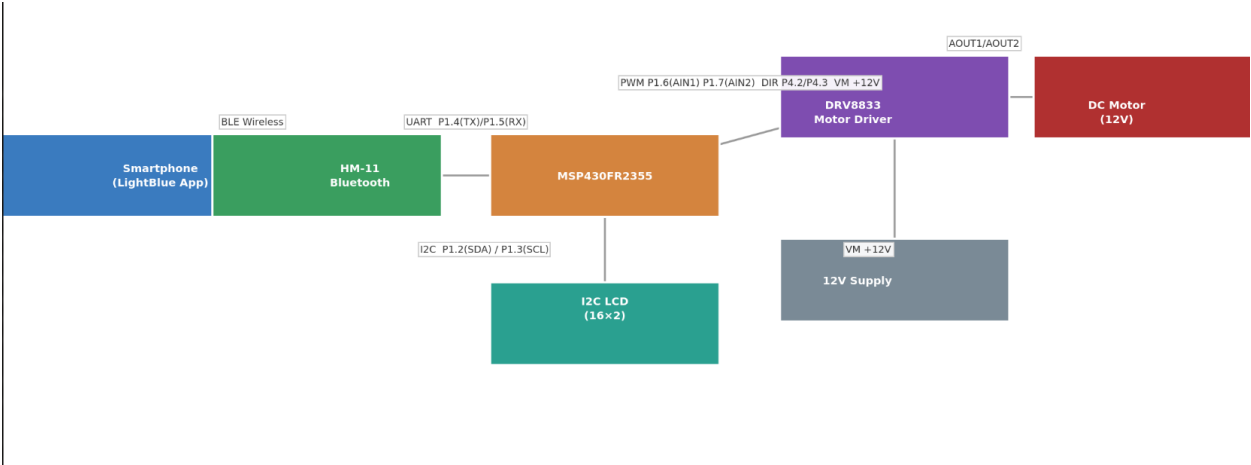
MSP430	LCD
P1.2	SDA
P1.3	SCL
5V	VCC
GND	GND

MSP430FR2355 → L298N Motor Driver

MSP430	L298N
P6.1	ENA
3.3V	IN1
GND	IN2
GND	GND

Power and Motor Connections

Source	Destination
Battery +	L298N 12V
Battery -	L298N GND
OUT1	Motor
OUT2	Motor



PWM Signal- Signal Control

PWM controls motor speed by adjusting the duty cycle of a digital signal. A higher duty cycle results in a higher average voltage applied to the motor, increasing its speed.

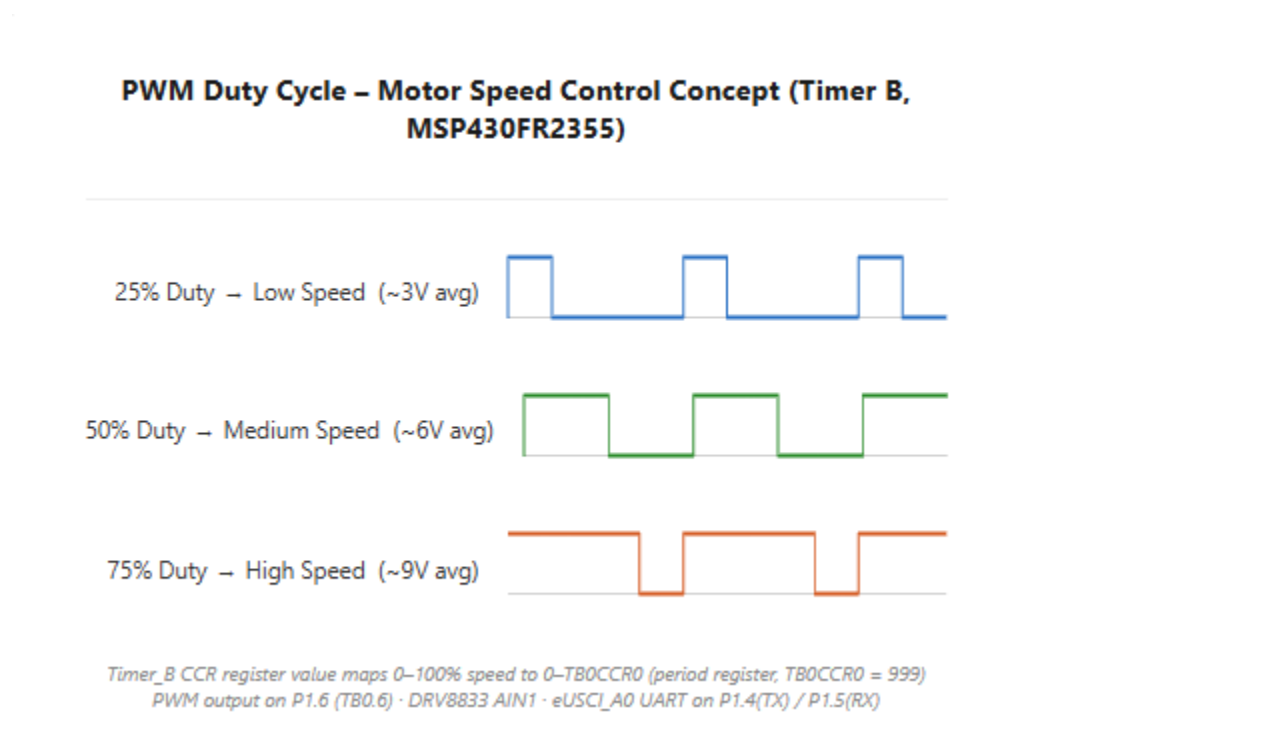
Timer_B3 is configured with:

TB3CCR0 = 655

The duty cycle is calculated as:

$TB3CCR2 = (\text{speed} \times 655) / 100$

This allows smooth and proportional control of motor speed.



Speed mapping formula used in code:

```
// Maps 0-100% to 0-1000 (TBCCR0 value)
uint16_t duty = ((uint32_t)percent * PWM_PERIOD) / 100;
TB0CCR6 = duty;
```

Smartphone Application

The LightBlue application is used to communicate with the HM-11 module. The module appears as a BLE peripheral (usually named "HMSoft").

The user sends commands in the format:

<number>#

Examples:

25#
50#
100#

The '#' symbol indicates the end of the input and triggers the motor update.

System Operation Summary

1. User enters a speed value on smartphone
2. Bluetooth sends command to HM-11
3. HM-11 transmits data to MSP430 via UART
4. MSP430 reads and processes input
5. PWM duty cycle is updated
6. Motor speed changes accordingly
7. LCD displays the updated speed

The table below traces a complete speed command from the smartphone through to the motor:

Step	Action	Component
1	User moves speed slider to 75% on the smartphone app	Smartphone App
2	App sends string "75\r\n" over Bluetooth RFCOMM	Smartphone → HM-11
3	HM-11 decodes BT packet; sends bytes to MSP430 via UART	HM-11 → P1.5 (RX)
4	UART RX ISR accumulates '7', '5', then detects '\r' terminator	MSP430FR2355 ISR
5	ISR calls atoi() → spd=75; wakes CPU from LPM0	MSP430FR2355 CPU
6	setMotorSpeed(75): TB0CCR6 = 750 (75% of 1000)	Timer_B0 PWM
7	P1.6 generates 75% duty cycle PWM waveform at ~1 kHz	MSP430 GPIO → DRV8833
8	DRV8833 drives motor with proportional average voltage	DRV8833 → DC Motor
9	Motor runs at ~75% of full speed	DC Motor (12V)
10	MSP430 echoes "75\r\n" back via UART → HM-11 → Smartphone	MSP430 → Smartphone
11	Smartphone app updates speed % display to show 75%	Smartphone App

Demo Link:

<https://youtube.com/shorts/gkLZ1EVtPWI?si=NOMhh9qwh7uQ7lum>

Results:

The wireless motor control system was successfully implemented and tested using the HM-11 Bluetooth module, MSP430FR2355 microcontroller, L298N motor driver, and a 12V DC motor. During testing, speed commands of 75%, 85%, 100%, and 55% were sent sequentially from the smartphone using the LightBlue application. The MSP430FR2355 correctly received each command through UART communication and updated the PWM duty cycle accordingly. The motor responded immediately to these inputs, with clearly observable changes in speed. At 75%, the motor operated at a relatively high speed, which increased further at 85%, and reached its maximum speed at 100%. When the input was reduced to 55%, the motor slowed down proportionally, confirming that the PWM mapping and motor control were functioning correctly. The I2C LCD displayed the speed percentage in real time and generally reflected the correct values after each command. Some minor inconsistencies in the LCD updates were observed after multiple rapid inputs, which were attributed to timing interactions between UART interrupts and display updates. This was addressed by adjusting the code to manage LCD updates more reliably. Overall, the system demonstrated accurate wireless communication, stable motor control, and effective real-time feedback.

Conclusion:

This project successfully achieved the goal of designing a wireless motor speed control system using the MSP430FR2355 microcontroller. By integrating Bluetooth communication through the HM-11 module, UART data handling, PWM signal generation, and I2C LCD interfacing, a complete embedded system was developed. The system allows a user to wirelessly control the speed of a 12V DC motor from a smartphone while simultaneously displaying the selected speed on an LCD. Throughout the implementation, several practical challenges were encountered, particularly related to synchronization between communication interrupts and display updates. Addressing these issues provided valuable insight into real-world embedded system behavior and debugging techniques. The final system demonstrated reliable performance, accurate speed control, and effective integration of multiple hardware components. This project reinforced key concepts such as peripheral interfacing, interrupt-driven communication, and hardware-based PWM control, and provided meaningful hands-on experience in building a complete embedded solution.